

Demo v6.2 流水线

Demo v6.2 流水线：25 个设计问题的代码答案

本文按设计评审中的 25 个问题，说明 Demo v6.2 每个阶段由哪个模块负责、数据如何流动，以及出错时在哪里终止。每个答案末尾都给出“源码证据”，引用格式为 文件::函数/类/方法；设计文档只作为语义补充，不替代实现证据。行号链接对应当前 single-camera 工作区源码。

整体运行结构如下：main.py::main 是总控入口，它启动 main_data_processing.py 子进程。后者构造 mdp/runtime.py 的 MainDataProcessingDemo，负责摄像头、分割、跟踪和 warm-up。总控进程持续读取摄像头进程写出的 frames.jsonl，通过 streaming/ 包生成在线 chunk，同时通过 shape_prior/ 包生成 SAM3D shape prior，最后启动一个下游消费者：visualization/visualize_track.py，或 Phystwin_shen 的 scripts/run_online_full_pipeline.py supervisor。Demo 只直接管理这个 supervisor；Stage 1、可选 Stage 2、train 和一个合并 HTML viewer 由外部 wrapper 创建并继承同一个进程组。生命周期事件写入 pipeline_status.jsonl；Q25 会区分“已经实现的状态写入”和“默认 viewer 尚未显示的部分”。

camera 子进程不再使用 mixin/共享 self: MainDataProcessingDemo 是一个 composition root，构造时显式接线四个 stage 类——CaptureStage (mdp/capture.py)、SegmentationStage (mdp/segmentation.py)、TrackerStage (mdp/tracker.py) 和 FormalProductStage (mdp/formal_products.py，承接 strict-pair 顺序发布与 row 落盘)——以及若干服务对象：CameraSession (mdp/session.py，相机 runtime/数据源/标定/ headless writer/depth engine)、ShapePriorPublisher (mdp/shape_prior_flow.py，frame-0 提交、packet 富化、warmup-finished 切换)、LosslessPipeline/FatalErrorLatch/FormalTimelineGate/StageStatsBoard (mdp/plumbing.py) 和 frozen RunMode (mdp/cli.py)。跨 stage 的共享依赖全部通过构造参数显式注入；原 annotation-only 的 typing contract 已随 mixin 一起删除，不再存在第二条声明路径或额外状态。

正式启动命令与当前实测边界

当前 config/default.yaml 的正式默认组合是：input_source=fake-live、data_process_base_path=outputs、downstream.mode=phystwin_shen。因此不带调试 override 的无 warm-up 窗口启动命令是：

```
conda run -n demo_2_max --no-capture-output \  
python demo_v6_2/main.py \  
--input-source fake-live \  
--no-warmup-rgb-preview
```

这条命令先写 <repo>/outputs/，在 shape_prior/points.npz ready 后启动 Phystwin_shen supervisor。外部 wrapper 随即启动 combined HTML viewer，默认地址是 <http://127.0.0.1:8765/>；页面开始监听后可用 Chrome 打开。浏览器本身不会启动 viewer server，所以应先看到 STAGE_DOWNSTREAM_START，或确认该 URL 返回 HTTP 200。

2026-07-12 做了两层验证。上游有界运行使用 --max-chunks 1 与 --downstream-mode disabled，进程 exit 0，提交 1 个 5 帧 chunk；shape prior 完整、track_process_status=normal、ASAP fallback frame 为 0。因为达到 max_chunks 后编排器主动终止仍在 replay 的 camera 子进程，summary 中 camera return code -15 是预期 SIGTERM，不是 camera failure。随后按上述正式命令启动完整下游：outputs/ 持续提交 chunk，points-ready trigger 启动 supervisor，viewer 返回 HTTP 200，Stage 1 读到在线 chunk 并导出首个 realtime candidate。这证明了正式启动到 viewer/Stage 1 的 live 路径，不把仍在运行的完整 5658 帧 replay 和 100-iteration train 误报为已经终态成功。精确命令与观测记录在 [2026-07-12-demo-v6-2-fake-camera-phystwin-proof.md](#)。

总体流水线与并发边界（Q1）

1. 总体 pipeline 是什么？各阶段是线程还是进程，为什么这样设计？正式 fake-live 与 live 使用同一张并发拓扑；差别只在 capture thread 的数据源是录制文件还是 RealSense。默认 phystwin_shen 下游可以概括为：

```

OS process P0: demo_v6_2/main.py (orchestrator 主进程)
  main thread:
    启动/监控 P1 -> tail frames.jsonl -> 组装/原子提交 chunk
    -> 等待 P2 结束 -> 写 run summary/status
  optional thread:
    Phystwin supervisor stdout/log relay
  |
+-- OS process P1: main_data_processing.py (camera/perception 进程)
  |   formal strict threads:
  |     capture -> seg -> processed-frame -> tracker -> pair-output
  |   auxiliary threads:
  |     shape-prior manager; 可选 warm-up RGB preview
  |   temporary OS processes:
  |     prewarmed upscale / SAM3D generate / align; cold sample
  |   output:
  |     capture/prepared_phystwin/*.npz + frames.jsonl
  |
+-- OS process P2: Phystwin_shen full-pipeline supervisor
    child processes:
      combined HTML viewer (与计算并发)
      Stage 1 -> optional Stage 2 -> train (按顺序执行)

```

P0 的 chunk bridge **不是线程池**：ChunkStreamSession.run 就运行在 orchestrator main thread，同步完成 tail、window tracking/ASAP、archive、chunk 和 manifest commit。这样 online stream 只有一个 writer 和一个 commit 顺序，不需要在多个 writer 间协调 chunk id/manifest。SameSeqPairer、OrderedPacketQueue 和 LatestSlot 也只是 P1 内的同步/缓冲对象，不是隐藏线程。

P1 的正式 masked path 使用 5 条 daemon thread。capture 生成 FramePacket；seg 维护一个 session-lived EdgeTAM state；processed-frame 对正式 5 FPS 帧执行 depth、camera-to-world、固定 PT mask refinement 和 runtime PCD；tracker 维护 session-lived TAPNext++ state；pair-output 只发布同 seq 的 PCD/tracker 结果。

_capture_recording_worker 名字里虽然有 worker，但它由 capture thread 同步调用，不是第 6 条 thread。无 tracker 的 profile-only 路径可以有不同 worker 列表，但不属于正式 masked pipeline。

为什么 hot path 用线程？ 这五个阶段需要共享模型/session、NumPy/Torch buffer、CUDA context 和 frame packet。线程让 packet 保持内存传递，避免每帧 pickle/copy，也避免为每阶段复制大模型。主要计算在 OpenCV/PyTorch/CUDA 中，不依赖纯 Python CPU 并行；bounded OrderedPacketQueue 提供 backpressure、连续 seq 和禁止静默覆盖。共享 CUDA 的代价是首帧编译/模型初始化可能互相干扰，所以 capture 在 frame 0

后等待首个完整 PCD/tracker pair，再释放 frame 1；任何 thread 异常都记录为 fatal 并设置统一 stop_event。

为什么重模型和下游用进程？ camera/perception、shape-prior、viewer 和 Phystwin 属于不同生命周期与故障域。进程边界可以：

- 给 camera 和 Phystwin 建立独立 process group，异常时杀掉全部 descendants；
- 让 SAM3D 临时子进程退出后真正释放 GPU 0，再把 GPU 0 交给 Phystwin；
- 让主 camera 热路径固定使用 GPU 1，避免大模型在同一 CUDA context 中残留；
- 允许外部 Phystwin checkout/CLI 保持自己的工作目录和 stage 生命周期；
- 隔离 GUI、模型加载或外部 stage crash，不让它直接破坏 camera 内存状态。

代价是 process startup 与磁盘交接更重，因此只有生命周期/资源边界使用进程；高频逐帧阶段留在线程内。P1→P0 用 prepared NPZ/JSONL，P0→P2 用原子 online_data/chunks + manifest 文件协议：它比内存 queue 慢，但能跨仓库、让生产者和训练侧解耦速率、保留可检查/可补读的 committed history。总体原则是：**同一实时状态和 CUDA hot path 用线程；需要独立 GPU 释放、故障清理、外部 runtime 或持久化交接的边界用进程/文件协议。**

目录门面遵循同一个边界原则：demo_v6_2/ 根目录只保留 Q2–Q25 直接点名的 Python facade/证据模块；只在前言/Q1 出现或只服务内部实现的模块分别进入 mdp/、orchestration/、streaming/、perception/、shape_prior/ 和 visualization/。没有在根目录保留 import forwarding wrapper；调用方必须使用唯一 canonical package path。

源码证据：

- `main.main` 用 `Popen(..., start_new_session=True)` 启动 P1，并在 main thread 运行 `ChunkStreamSession.run`。
- `MainDataProcessingDemo` 给出 composition root 对四个 stage 与各服务对象的接线（`main_data_processing.py` 只是子进程入口 facade）；
`MainDataProcessingDemo._start_threads` 是 P1 正式 thread 列表的唯一创建点。
- `OrderedPacketQueue`、`SameSeqPairer` 与
`CaptureStage._publish_capture_packet` 给出内存队列和 seq 合同。
- `ShapePriorWarmupManager.maybe_submit` 创建 manager thread；
`ShapePriorLocalClient` 创建/调用各 stage 子进程；`WarmupRgbPreview.start` 创建可选 preview thread。
- `launch_phystwin_shen` 创建 P2 和 output-relay thread；外部
`scripts/run_online_full_pipeline.py` 创建 viewer/stage children。

摄像头与逐帧 I/O (Q2-Q7)

2. 摄像头在哪里启动？正式编排路径中，`main.py::main` 先调用 `main_subprocess.build_main_data_processing_command`，再用 `subprocess.Popen` 启动 `main_data_processing.py`。子进程入口构造 `mdp/runtime.py` 的 `MainDataProcessingDemo`；其 `run` 调用 `CameraSession.prepare_source` (`mdp/session.py`)，后者按输入模式选择 `mdp.capture_source._start_realsense_pipeline` (`live`) 或 `RecordedRgbdfFrameSource` (`fake-live`)。

正式编排的设备由 `config/default.yaml` 中 `camera.camera_serials` 指定，默认是 `["239222300740"]`。列表 `schema` 为以后多相机扩展保留，但 `main_options.resolve_camera_serials` 当前要求 恰好一个非空 `serial`；可重复的 `--camera-serial` 会整体覆盖配置列表。

`main_subprocess.build_main_data_processing_command` 将唯一 `serial` 传给 子进程 `--serial`，`_start_realsense_pipeline` 再执行 `config.enable_device(serial)`。`table_calibrate_metadata.json` 也记录该 `serial` 为 `table-calibration reference`。

这个“一台指定相机”的约束只覆盖由 `main.py` 启动的正式路径。若直接运行 `main_data_processing.py` 且不传 `--serial`，`utils.camera.resolve_serial` 仍会选择排序后的第一个 D400。此前 D405 缺少所需 RGB profile 的报错是实机验证结果，不是 Python 源码主动抛出的 固定错误。

源码证据：

- `main.main` → `build_main_data_processing_command` → `main_data_processing.main` → `MainDataProcessingDemo.run`。
- `CameraSession.prepare_source` 包含 `replay/live` 分支；`_start_realsense_pipeline` 绑定设备并 启动 RealSense pipeline。
- `resolve_camera_serials`、`main_cli.build_parser`、`camera.camera_serials` 和 `table_calibrate_metadata.json` 共同 证明正式路径的 `serial` 合同。
- `list_d400_serials` 返回排序列表，`resolve_serial` 在未指定时取 `serials[0]`；实机 结果记录在 `2026-07-09-demo-v6-2-refactor.md`。

3. 摄像头线程在哪里创建？`MainDataProcessingDemo._start_threads` 组装 worker 列表，并统一创建 `daemon=True` 的 `threading.Thread`。正式 `strict` 路径包含 `capture`、`seg`、`processed-frame`、`lossless tracker` 和 `pair-output worker`。

`pcd_mode=masked` 只允许这条 `strict` 路径：参数校验要求 `TAPNext++ tracker` 且 `track_mode != none`。无 `tracker` 只允许 `pcd_mode=none` 的纯 `capture/depth isolation`；旧的 `latest-frame _pcd_worker` 已删除。`processed-frame worker` 对正式 5 FPS 帧建立完整 `world-space dense grid`，按 $0.2 < \text{depth} < 1.5 \text{ m}$ 与固定 `PT radius rule` 清理二维 `mask`，再从同一份 `processed mask` 选择 `runtime PCD`。它不执行点数 `cap`、`table-Z` 删除或 `mask erosion`；`prepared writer` 只打包该结果，不再次运行 `PT`。

seg 和 lossless tracker 会在同一个进程、同一个 CUDA device 上并发执行。EdgeTAM 的 reduce-overhead 会在第 2 次 model call 录制 CUDA graph；TAPNext++ 第一次拿到 mask 时才构造 CUDA model，并在参数初始化时使用 RNG。为避免两者重叠，live 与 replay 都会在发布 frame 0 后等待完整的首个 PCD/tracker pair，再释放 frame 1。这个启动 handshake 保留后续并发与 compile 性能，同时满足 CUDA graph capture 期间不能有其他线程 CUDA 工作的约束。_capture_recording_worker 不是另一条线程；replay 模式下，capture 线程进入 CaptureStage.run 后同步调用它。

shape-prior warm-up 另由 ShapePriorWarmupManager.maybe_submit 创建一条 shape-prior-warmup daemon thread；它不在 MainDataProcessingDemo._threads 列表中。

源码证据：

- `MainDataProcessingDemo._start_threads` 定义 worker targets，并在一个循环里调用 `threading.Thread(..., daemon=True)`。
- `CaptureStage.run` 的 replay 分支 直接调用 `_capture_recording_worker` 后返回。
- `ShapePriorWarmupManager.maybe_submit` 单独创建 shape-prior warm-up 线程。

4. **进程和线程如何协作？** 外层是多进程：总控进程启动 camera 子进程；可选 visualizer、Phystwin_shen full-pipeline supervisor 也是总控的直接子进程。supervisor 再按配置启动一个合并 HTML viewer、Stage 1、可选 Stage 2 和 train；这些 child 不创建新 session，因此继承 supervisor 的进程组。shape-prior 不是一条与 camera 并列、常驻的单一进程：camera 进程先启动 warm-up thread，ShapePriorLocalClient.request_shape_prior 再顺序调用各阶段子进程。

camera 进程内部的 strict 数据流是：capture 同时写 capture_slot 和 LosslessPipeline.frame_queue；seg 将 raw mask 写到唯一 mask_queue；processed-frame worker 完成 depth/world-grid/PT 后，把同一 canonical frame 交给 runtime PCD 与 processed_frame_queue 中的 tracker。PCD 与 tracker 结果由 SameSeqPairer 按相同 seq 配对，再进入 pair-output queue。PairedBuildResult.__post_init__ 在对象构造边界再次要求 PCD packet、其 source mask、tracker packet 和 pair 本身四个 seq 完全相同；旧 PairedRenderPacket/paired_render_slot 中转层已经删除。pair-output worker 直接发布验证后的 build result。OrderedPacketQueue 保序且不静默覆盖，LatestSlot 只保留最新值。stop_event 管理 MainDataProcessingDemo._threads 中的 workers；shape-prior manager thread 不检查该 event，因此不能笼统说“所有线程都由 stop_event 退出”。

源码证据：

- `main.main`、`launch_phystwin_shen` 和 `ShapePriorLocalClient.request_shape_prior` 给出 Demo 侧进程边界；full-pipeline wrapper 的 child 顺序由外部 `scripts/run_online_full_pipeline.py` 定义；shape-prior 阶段冷启动由 `_run_stage` 调用 `subprocess.run`。

- `CaptureStage._publish_capture_packet`、`OrderedPacketQueue`、`SameSeqPairer` 和 `LatestSlot` 给出实际的线程间数据合同。
- `PairedBuildResult.__post_init__` 是 strict pair 的最终同序号校验；stage/服务间的共享依赖由 `MainDataProcessingDemo.__init__` 以构造参数显式注入，不再有单独的 typing contract。
- `MainDataProcessingDemo.stop` 设置 `stop_event` 并 join `_threads`；shape-prior thread 的入口是 `ShapePriorWarmupManager._run`。

5. **RealSense RGB 和 depth 的 FPS 是多少？** 由 `main.py` 启动的正式路径默认以 **30 FPS** 采集 RealSense。 `build_main_data_processing_command` 把外层 `--camera-fps` 传成子进程 `--fps`，`_start_realsense_pipeline` 对所有启用的 color、IR 或 depth stream 都使用同一个 `int(args.fps)`。

strict live 输出采样由 `LiveLatestFrameSampler` 控制：直接输入是子进程的 `--lossless-input-fps`，外层由 `resolve_camera_source_replay_fps` 解析。未单独设置 `--camera-source-replay-fps` 时它回落到 `--replay-fps`，默认 **5 FPS**；设置 override 后，camera 取样频率可以与 chunk/metadata 使用的 `replay_fps` 分开。采样器每隔 $1/\text{fps}$ 发布当时的最新帧。直接运行 `main_data_processing.py` 的内层 `--fps` 默认仍是 60，因此 30 FPS 结论应限定为正式 `main.py` 编排默认。

这层采集频率不等于发布/物理频率。Demo 的 `replay_fps` 固定输出 **5 FPS** 均匀时间步；Phystwin_shen `configs/real.yaml` 同样使用 `FPS: 5`、`dt: 5e-5`、`num_substeps: 4000`，即每个数据帧模拟 $5e-5 \times 4000 = 0.2 \text{ s} = 1/5 \text{ s}$ 。因此不存在原先 30 FPS 物理时间比 Demo 快 6 倍的问题；RealSense 仍可在 30 FPS 获取最新输入。

PT mask refinement 同样位于这个 5 FPS formal 边界之后，所以 30 FPS 相机帧只负责提供最新输入，不会逐帧承担 dense world-grid/PT 计算。

源码证据：

- `config/default.yaml` 定义 `replay_fps: 5.0`，`camera.camera_fps` 定义 `camera_fps: 30`；`build_main_data_processing_command` 将后者传给子进程。
- `_start_realsense_pipeline` 对每个启用 stream 都传 `int(args.fps)`。
- `resolve_camera_source_replay_fps`、`CaptureStage.run` 和 `LiveLatestFrameSampler` 证明“固定 tick 取最新帧”的实际控制链。
- `mdp.cli.build_parser` 与 `mdp.constants.DEFAULT_FPS` 证明直接运行子入口的默认值是 60，而不是 30。

6. **每帧在哪里读取？** live 模式由 `CaptureStage.run` 调用 `pipeline.wait_for_frames()`；native-depth 路径先 `align.process`，FFS 路径读取 color 与左右 IR。replay 模式由 `_capture_recording_worker` 调用 `RecordedRgbDFrameSource.read_packet` 加载录制 RGB-D/IR 文件。两条路径都生成 `mdp.packets.FramePacket`。fake-live 的 `make_runtime` 保留相机几何和序列号，但令

`RealtimeCameraRuntime.pipeline=None`；只有 live runtime 持有需要停止的 `RealSense pipeline`。

`_publish_capture_packet` 总会写 `capture_slot`；正式 strict 模式还会同时写 `lossless_frame_queue`，不是二选一。

源码证据：

- `CaptureStage.run` 包含 `RealSense` 读取、对齐、数组复制和 `FramePacket` 构造。
- `_capture_recording_worker` → `RecordedRgbFrameSource.read_packet` 是 replay 读取链。
- `RecordedRgbFrameSource.make_runtime` 和 `RealtimeCameraRuntime` 定义 fake-live 的无硬件 runtime 合同。
- `FramePacket` 定义逐帧包；`CaptureStage._publish_capture_packet` 明确先写 slot，再在 lossless 模式写 queue。

7. **frame id 和 timestamp 从哪里来？** `FramePacket.seq` 是 Demo 内部逻辑序号。live 路径用 `output_seq` 从 0 连续重编号；replay 路径把 `runtime_seq` 作为内部 seq。

fake-live 的来源字段更细：`source_timestamp_s` 来自录制 metadata 的 value；`source_frame_index` 是排序后 frame refs 中被选中的位置；原始 metadata key 则保存在 `source_step`。当前 live 实现没有调用 `RealSense`

`get_timestamp()/get_frame_number()`，所以 live `FramePacket` 的三个 `source_*` 字段保持 `None`。原文“真实采集模式下来自摄像头”不受当前源码支持。

有值的 `source_*` 会一路写入 prepared frame 和 archive mapping。正式均匀 时间轴使用连续 `online/output index` 除以 `fps`，而不是 `raw source_frame_index / fps`；`source_*` 只做 provenance。

源码证据：

- `CaptureStage.run` 的 `publish_output_packet` 连续重编号，且 live `FramePacket(...)` 未赋 `source_*`。
- `RecordedRgbFrameSource._build_frame_refs` 与 `read_packet` 分别构造 step/timestamp，再写 `source_timestamp_s/source_frame_index/source_step`。
- `HeadlessCaptureWriter.write_pcd` → `prepare_phystwin_frame` 保留 provenance；`OnlineFrameArchive.archive_chunk` 写 online-to-source mapping。
- `ChunkDataWriter.commit_chunk_data` 维护连续 `start_frame/end_frame`；`_materialize_and_commit_window` 用 `row_start/fps` 和 `row_end/fps` 生成均匀窗口时间。

Warm-up (Q8–Q15)

8. Warm-up 使用单帧还是一段视频？ 初始化使用一张帧。

SegmentationStage._prepare_warmup 等到一张 first_frame，只把这张图交给 SAM3.1 (mdp.warmup 的 frame-0 seed)，生成 object、hand A、hand B 的 InitialMaskBundle。随后 SegmentationStage.run 创建一个 EdgeTAM session，并对同一 first_frame 调用 _run_segmentation_frame(..., add_prompt=True)。这是一个 session 中的三个 identity，不是三个 session。

源码证据：

- SegmentationStage._prepare_warmup → run_sam3l_first_frame_mask_bundle 只处理 first_frame.color_bgr。
- SegmentationStage.run 创建一个 EdgeTamVideoInferenceSession，再调用 _run_segmentation_frame 并设置 add_prompt=True。

9. 系统如何确认拿到的是 frame 0？ 正式 strict 路径不是靠 sentinel 单独保证，而是靠 producer handshake 加 FIFO：live 首次发布时 output_seq == 0，replay 首次显式调用 read_packet(seq=0)；两者在发布首帧后都会等待 _first_frame_segmented，随后继续等待 LosslessPipeline.first_pair_published，不会 先把后续正式帧灌入。strict _wait_for_first_frame 从 OrderedPacketQueue 队首取帧，因此得到 seq 0。

非 strict 分支才调用 capture_slot.get_latest_after(-1)；它只保证返回 seq > -1 的当前最新帧，不能单独证明一定是 seq 0。首帧只“添加 prompt 一次”，但它还会继续用于 PCD、tracker、shape prior 和 warm-up anchor，不能说整张帧只用于初始化。

源码证据：

- SegmentationStage._wait_for_first_frame 展示 strict queue 与 non-strict latest-slot 两条分支。
- CaptureStage.run 和 _capture_recording_worker 都先发布 seq 0，再 等待首帧 分割 handshake。
- OrderedPacketQueue.put/get 保持连续 FIFO；LatestSlot.get_latest_after 则是 latest-wins。
- SegmentationStage.run 对首帧用 add_prompt=True，后续帧统一用 False。

10. Warm-up 期间后续到达的帧如何处理？ 后续帧继续进入同一个 EdgeTAM session，使用 add_prompt=False，并继续生成 mask、PCD、tracker 结果和 input preview。只有正式 product row 受 gate 控制：chunk-ready anchor 已写且 shape-prior 仍为 pending/running 时，FormalTimelineGate.rows_gated 扣留 frames.jsonl row 和对应 tracker sidecar。

“期间 frames.jsonl 只有 frame 0”需要限定：anchor 前可能先写入无效 startup rows，chunk bridge 随后会修剪；gate 超时或 shape-prior 进入终态后也会解除。因此这句话只适用于正常

的“anchor 已写、prior 仍在运行、gate 未超时”区间。

源码证据：

- `SegmentationStage.run` 对后续帧调用 `_run_segmentation_frame(..., add_prompt=False)` 后仍发布 mask。
- `FormalProductStage._publish_strict_pair` 在 PCD/tracker 已配对后只对落盘行应用 gate；`_formal_timeline_rows_gated` 定义其状态条件。
- `FormalProductStage._write_headless_pcd_result` 在 gated 时 返回；`HeadlessCaptureWriter.write_input_frame` 不经过该 gate。
- `WarmupStartFilter._trim_unready_startup_rows` 修剪无效 startup rows；`FormalTimelineGate.rows_gated` 实现超时 解除逻辑。

11. **Warm-up 最耗时的步骤是什么？** 权威产物是 `capture/shape_prior_profile.json`。现在它把操作员等待时间拆成 三层，而不是只报告 submit 后的五个粗阶段：

1. `shape_prior_timing.pre_submit`: camera runtime start → frame 0 receive → EdgeTAM mask ready → PCD ready → shape-prior submit。它同时保留 frame-0 SAM3.1、EdgeTAM 和 PCD 的已有细分 timing。
2. `shape_prior_timing.critical_path`: case write → upscale → SAM3.1 RGBA export → SAM3D generate → align → sample → result finalize。每项有 start/end offset、wall duration 和子阶段 details。
3. 顶层 `warmup_shape_prior_ready_to_gate_open_ms` 与 `warmup_total_ms`：记录 prior ready 后写 capture 产物并真正打开 formal gate 的延迟，以及从 camera runtime start 到 gate open 的总等待。

当 shape-prior 产物已经写入且 formal gate 首次打开时，camera 子进程会在 当前终端只打印一次醒目的 Warmup finished banner。它和 `STAGE_WARMUP_READY` 在同一个真实完成边界触发，不会在 submit 或后台计算 尚未完成时提前显示。

`shape_prior_timing.ranking` 按关键路径 wall duration 排序，bottleneck 是本轮第一优化目标；`accounted_ms` 和 `unattributed_ms` 用于检查计时是否 闭合。这里的排名只描述本轮，不能用单次结果代替多轮 p50/p95。

三个预热子进程还会在 `<shape-prior-case>/shape/timing/{upscale,generate,align}.json` 写 READY 与 completed 快照；sample 写 `sample.json`。父进程比较 READY wall time 与 GO wall time，给出 `ready_before_go`、`ready_lead_ms` 和 `startup_tail_on_critical_path_ms`，因此可以区分“模型已经预热完成”和“GO 发出后仍在补模型加载”。

`profile_snapshot_to_parent_return_ms` 还把 最后一次 profile snapshot 后的 JSON flush/进程退出成本单独暴露出来。

子阶段拆分直接对应可优化动作：upscale 区分 model load/crop/inference/ PNG write；generate 区分 prepare/model load/pipeline run/GLB/PLY export；align 区分 input/render candidates/SuperGlue/PnP+scale/两段 ARAP/mesh export；sample 区分 mesh load/surface+volume sample/voxel dedup/pickle。

generate 的 SAM3D 调用固定关闭 with_layout_postprocess：Demo v6.2 只消费 GLB mesh 和 Gaussian，不使用 SAM3D 返回的 layout/pose 字段；随后独立的 shape_prior/align.py 才负责把 mesh 配准到 frame-0 观测。因而 layout pose 后处理不会决定最终对齐，只会增加 warm-up 关键路径。mesh postprocess 与 texture baking 仍保持启用，分别服务后续 mesh 对齐/采样和带纹理 GLB 导出。

2026-07-12 正式 fake-live 运行生成了新 schema 的完整 profile：camera runtime start 到 shape-prior submit 为 16.186 s；submit 后关键路径为 58.511 s；prior ready 到 formal gate open 仅 75.8 ms；总 warm-up 为 74.773 s。关键路径排名是 generate 29.415 s (50.27%)、align 14.139 s (24.16%)、upscale 11.225 s (19.18%)、sample 3.133 s (5.35%)。前三项 合计 93.62%，generate 仍是第一优化目标。

本轮三个长阶段都确认 ready_before_go=true 且 startup_tail_on_critical_path_ms=0，说明预热 worker 已在 GO 前完成启动，当前瓶颈不是迟到的 worker cold start。generate 的 pipeline_run_ms 约 14.567 s；align 中 render_candidates_ms 约 7.366 s、superglue_match_ms 约 4.081 s，明显高于两段 ARAP 合计约 1.213 s。因而下一轮优化应优先看 generate pipeline/export，再看 align 的 render/match，而不是二次 SAM3.1 或 ARAP。

源码与实测证据：

- [shape_prior.timing](#) 定义 schema 校验、关键路径 闭合与 bottleneck 排名；[ShapePriorLocalClient.request_shape_prior](#) 聚合它。
- [run_sam3d_shape_prior](#)、[shape_prior.align.main](#)、[shape_prior.sample.main](#) 和 [shape_prior.upscale.main](#) 写子进程明细。
- [2026-07-12-demo-v6-2-fake-camera-phystwin-proof.md](#) 固化了本轮命令、profile 数字、viewer HTTP 检查和验证边界。

12. **Warm-up 完成后保留哪些状态和文件？** seg worker (`SegmentationStage.run`) 在其生命周期内保留 `SegmentationWarmupState`、`InitialMaskBundle` 和唯一的 `EdgeTAM session`；三个 identity 在同一次 `add_inputs_to_inference_session` 中注册。shape-prior manager 成功后还在 `self._result` 中保留 `ShapePriorResult`。

磁盘上会保留 offline-style shape-prior case、配置路径下的 `shape_prior/points.npz`、capture 下的 `shape_prior/points.npz`、`<case>/shape/matching/final_mesh.glb`，以及 shape-prior sampling 生成的 `<case>/final_data.pkl`。

源码证据：

- `InitialMaskBundle` 与 `SegmentationWarmupState` 的实例由 `SegmentationStage.run` 持有；同一 worker 只构造一个 session。

- `ShapePriorWarmupManager._run` 成功时写 `self._result`, `ready_result` 读取它。
- `shape_prior/case.py` 是 frame-0 case 序列化模块: `write_shape_prior_case` 与 `ShapePriorLocalClient.request_shape_prior` 写 case 和配置的 `points.npz`; `HeadlessCaptureWriter.write_shape_prior_result` 写 capture 副本。
- `shape_prior.align.main` 导出 `final_mesh.glb`; `shape_prior.sample.main` 写 case `final_data.pkl`。

13. **Warm-up 状态如何校验?** `SegmentationStage._prepare_warmup` 在 SAM3.1 返回后校验 `controller/object mask` 与输入帧尺寸; `_union_masks` 拒绝“没有任何 instance mask”和同一 label 内形状不一致, 但不会把“一张全 false mask”自动视为缺失。
`split_controller_hand_instances` 必须得到两只非空、可分离的手。

`canonical processed-frame` 边界校验 `c2w`, 并在固定 PT 后立即拒绝空 object 或 `controller mask`。`write_shape_prior_case` 只校验并写入同一份 `cleaned mask/dense grid`, 不重算 PT, 也不会借用 object point 伪造 `controller`。

源码证据:

- `SegmentationStage._prepare_warmup`、`_union_masks` 和 `split_controller_hand_instances` 给出 frame-0 mask 校验。
- `FormalProductStage._build_processed_frame_result` 构建 canonical frame 并对空类别 fail fast; `write_shape_prior_case` 直接消费该 frame。
- `_wait_for_shape_points` 只检查 `surface+interior` 总数是否大于 0; 不存在更高的通用最小点数门槛。

14. **Warm-up 出错后如何处理?** 冷启动 `shape-prior stage` 由 `_run_stage` 使用 `subprocess.run(..., check=True)`; 预热 worker 也会在非零 return code 时抛 `CalledProcessError`。但 `shape-prior` 总控本身是 camera 进程内的 daemon thread; `ShapePriorWarmupManager._run` 会捕获 stage 异常, 将 profile 置为 failed, 而不是直接走 camera worker 的 fatal hook。

`segmentation` 等正式 workers 的异常会进入 `FatalErrorLatch.record`: 记录第一条 fatal、写 `fatal_error` 状态、设置 `stop_event`, 最后让 `MainDataProcessingDemo.run` 返回 2。`shape-prior` 失败时, `status` 不再是 `pending/running`, `row gate` 解除; 失败 profile 写入 capture metadata, `chunk bridge` 的 `_wait_for_shape_points` 看到 failed 后立即抛错, 避免无限等待。

源码证据:

- `_run_stage` 和 `PrewarmWorkerPool.pop_and_go` 给出子阶段非零 退出的异常路径。
- `ShapePriorWarmupManager.maybe_submit` 创建 thread; `ShapePriorWarmupManager._run` 捕获 异常并写 `STATUS_FAILED`。

- `FatalErrorLatch.record` 与 `MainDataProcessingDemo.run` 给出 camera worker 的 fatal/exit 路径。
- `_formal_timeline_rows_gated`、`ShapePriorPublisher.maybe_write_headless_result` 和 `_wait_for_shape_points` 给出 shape-prior failed → metadata → bridge exception 的真实传播链。

15. **正式时间线从哪里开始？** 成功路径中，第一个 chunk-ready row 占据 online/output frame 0 的 warm-up anchor；anchor 后、shape prior 尚为 pending/running 的 rows 被扣留。shape prior 进入 READY 后第一条未被 gate 的 row 紧接为 online frame 1。

OnlineFrameArchive 连续编号，同时在 `enhance_metadata.json` 保留原 seq 和 source mapping。

这里也有失败边界：gate 会在 timeout 或 failed/disabled 终态解除，所以“READY 后成为 frame 1”只描述成功路径；失败路径随后应由 bridge 报错。

源码证据：

- `_formal_timeline_rows_gated` 定义 anchor 后的 gate；`FormalProductStage._write_headless_pcd_result` 只让 controller ≥ 30 且 object > 0 的 row 取得 anchor，并记录首次 formal seq。
- `WarmupStartFilter._trim_unready_startup_rows` 保留首个 chunk-ready row；`OnlineFrameArchive.archive_chunk` 用 `online_start_frame + local_index` 连续编号。
- `design_spec.md` 记录 frame 0/1 接缝与 hold-still 约定，但实现依据是上述函数。

Chunk 组装、跟踪与过滤（Q16–Q20）

16. **Chunk 如何组装？** `ChunkStreamSession.run` 每读到一条 `frames.jsonl` row，就用 `_prepared_frame_from_row` 加载它引用的 canonical prepared NPZ，并将 row/frame 同步追加到两个 buffer。达到 `chunk_size` 后先形成 `pending_window`；下一条 row 通常作为 borrow/lookahead frame 到达后才 materialize，capture 结束时则无 borrow flush。borrow 只参与上一窗口尾帧的 motion verdict，不进入其发布数组。

`_chunk_data_window_from_prepared_frames` 校验所有帧共享 query schema，逐帧收集 processed mask，并将 track、visibility、PCD points/colors 沿时间维 stack。RGB 不在此处 stack，而是稍后由 `OnlineFrameArchive.archive_chunk` 逐帧写 PNG；`query_points_yx` 只做共享一致性校验。当前路径不会从 legacy sidecar 重建数据。

源码证据：

- `ChunkStreamSession.run` 负责 row/frame 双 buffer、`pending_window` 和 borrow-row 触发；配套的 `_materialize_pending` 是同一 session 的方法。

- `_prepared_frame_from_row` 要求 `prepared_phystwin_frame_path`；缺失或文件不存在立即失败。
- `_chunk_data_window_from_prepared_frames` 检查 query 一致性、收集 mask，并 stack track/visibility/PCD；`_materialize_and_commit_window` 将 RGB-D 交给 archive。

17. Chunk 大小在哪里配置？ `OrchestratorRunConfig.from_args`

(`orchestration/run_config.py`) 优先使用 显式 `--chunk-frame-count`；否则计算 `round(replay_fps × chunk_seconds)`，并要求结果大于 0。默认配置为 `5 FPS × 1 秒 = 5 帧`。结果传给 chunk stream，并保存为 `ChunkDataWriter.chunk_size`；writer 再次校验正数，并把它写入 manifest 和 static metadata。

源码证据：

- `config/default.yaml` 定义 5 FPS，`chunking.chunk_seconds` 定义 1 秒；`main_cli.build_parser` 暴露 override。
- `OrchestratorRunConfig.from_args` 实现 override/乘法/正数 校验；`main.main` 把 `config.chunk_frame_count` 传入 chunk stream。
- `ChunkDataWriter.__init__` 再次校验并保存 `self.chunk_size`；`_write_manifest` 发布它。

18. Chunk 按时间还是按帧数关闭？ 窗口边界严格按 **row/frame 数量**：每次 append 后，buffer 未滿就 continue，达到 `chunk_size` 才关闭。`window_closed_wall_s` 只是遥测，不参与边界判断。live 路径关闭完整窗口后，通常仍需等下一帧作为 borrow 才 materialize/commit，因此“按帧数关闭”不等于“第 `chunk_size` 帧到达 即发布”。

源码证据：

- `ChunkStreamSession.run` 用 `len(row_buffer) < chunk_size` 判断是否继续累积。
- 该 session 的 `pending_window` 与 `_materialize_pending` 证明 borrow-row 发布延迟；`_materialize_and_commit_window` 只把 `window_closed_wall_s` 写入 telemetry。

19. Chunk 组装后如何执行跟踪？ 唯一的实时 chunk-stream 入口为整个 session 创建一次 `tracking.TrackingRuntime`，并把同一实例传过 `_materialize_and_commit_window` 到 `_chunk_data_window_from_prepared_frames`。后者调用 `_track_input_with_session_query_schema` → `streaming.window_observations.build_window_observations` → `TrackingRuntime.process_window`。

chunk 0 的 `_freeze_identity` 冻结 controller anchors、object columns、query schema 和 neighbor table；后续窗口由 `_check_frozen_identity` 校验。`process_window` 用 `~ctrl_usable` 表示语义上的 temporary-invalid（没有单独命名的状态数组），再调用

`_recover_anchor` 和 `Kabsch_rigid_transform` 做局部刚性恢复。若单独调用 `window builder` 而不传 `runtime`，它会临时新建实例；“整个 session 一个 runtime”只保证在公开 stream 主路径中成立。

`track_process_status` 是结果质量 telemetry，不是另一个分支：只要当前 window 有 controller anchor 使用局部刚性 proxy，状态就是 degraded；否则是 normal。degraded chunk 仍会提交，proxy 事实另由 `controller_proxied` 和逐 anchor status 保留；schema/query/非有限值等合同违反仍然 fail fast。2026-07-12 的 1-chunk 有界验证为 normal，长 replay 后段观察到 degraded，符合这个非致命定义，不能把它表述为 worker failure。

源码证据：

- `ChunkStreamSession.run` 所属 session 在构造时创建一个 `TrackingRuntime`。
- `_track_input_with_session_query_schema` 调用 `build_window_observations`；`_chunk_data_window_from_prepared_frames` 调用 `TrackingRuntime.process_window`。
- `_freeze_identity`、`_check_frozen_identity`、`_recover_anchor` 和 `_rigid_transform` 给出冻结与恢复细节。
- `TrackingRuntime.process_window` 仅在存在 proxied anchor 时写 `TRACK_STATUS_DEGRADED`；`main.main::on_chunk_written` 只将该值写入 chunk-committed status event。

20. 跟踪后还会做哪些过滤？`tracking.motion_consistency` 执行动作一致性过滤：半径 0.01 m、至少 5 个邻居（radius query 未排除自身）、相似阈值 0.005 m，至少 50% 邻居同意。

depth-validity **mask refinement** 与固定的 3D radius-outlier mask refinement

（radius=0.01 m、nb_points=40）在 processed-frame worker 中按顺序执行一次；tracker、runtime PCD、shape prior、prepared writer 和 chunk 侧只消费同一结果，不再次做 radius refinement。不过

`streaming.window_observations.build_window_observations` 仍会在 track pixel 采样时重新计算逐 query 的 depth-valid，因此不能泛称“所有 depth-validity 判断只执行一次”。

tracking 之后，`AsapRuntime.augment_window` 以 visibility、motion validity、finite 和 nonzero 联合判定无效 object 条目并回填。`_deform_frame` 在约束太少或结果非有限时复用上一帧 mesh vertices，这就是 `design_spec_v6_1.md` 所保留的 silent-freeze 行为。

源码证据：

- `tracking.py` 的 `MOTION_*` 常量与 `motion_consistency` 给出 0.01/5/0.005/50% 规则。

- `FormalProductStage._build_processed_frame_result` 依次调用 `apply_depth_validity_to_mask_frame` 和 `apply_radius_outlier_to_mask_frame`; `prepare_phystwin_frame` 只验证和打包; `build_window_observations` 另做 query-level depth-valid。
- `AsapRuntime.augment_window` 计算 `valid_now` 并回填; `AsapRuntime._deform_frame` 实现 silent freeze。

训练侧 schema、manifest 与读取起点 (Q21–Q23)

21. 训练侧会收到什么数据? Demo 生产端把每个窗口写成

`online_data/chunks/chunk_{id:06d}.pkl`。固定 metadata 包括 `case_name`、`chunk_id`、`start_frame`、`end_frame`、`source_frame_indices`; `source_timestamps_s` 仅在有价值时写入。 `streaming.data_keys.REQUIRED_TIME_KEYS` 定义五个生产端必需时序键: `object_points`、`object_colors`、`object_visibilities`、`object_motions_valid`、`controller_points`。生产端通用 schema 把 `asap_surface_points`、`asap_interior_points`、`controller_proxied` 等列为 optional; 并不存在名为 `recovery_mask` 的字段。

但当前配置的 `Phystwin_shen consumer` 会把 `asap_surface_points/asap_interior_points` 提升为必需字段; 默认 `asap_augment=True` 会生产它们。因此“ASAP 可选”只描述 Demo writer 的通用 schema, 不描述当前 trainer 的更窄合同。

同一 session 还生成 `online_data/color/0/{k}.png`、`online_data/depth/0/{k}.npy` (uint16 mm)、`calibrate.pkl`、`metadata.json`、`enhance_metadata.json`, 以及前缀聚合后的 `data/final_data.pkl`。相机 metadata 只发布当前消费者实际读取的 `intrinsics`、`WH`、`fps`、`frame_num`、`serial_numbers`; 其中前两个由 `PhysTwin online optimizer/trainer` 读取, `fps` 由 `viewer` 读取, 后两个也保留原始 `PhysTwin case loader` 合同。 `enhance_metadata.json` 只保留诊断工具读取的 `frame_mapping`, 每条 mapping 只含 `online index`、`seq`、`source frame index` 和 `depth path`。

源码证据:

- `REQUIRED_TIME_KEYS/OPTIONAL_TIME_KEYS` 定义生产端键; `build_online_chunk_record` 定义 chunk metadata 与 `TIME_KEYS` 切片。
- `ChunkDataWriter.commit_chunk_data` 写 `chunk_{id:06d}.pkl`; `_append_static_data` 聚合 `data/final_data.pkl`。
- `OnlineFrameArchive._archive_one_frame`、`_initialize_calibration`、`_write_metadata` 和 `_write_enhance_metadata` 给出 RGB-D archive 布局。
- 当前外部 checkout 5b8c071 的 `OnlineFrameBuffer._validate_chunk_shapes` 将两个 ASAP key 列为 required; `main_cli.build_parser` 把 `asap_augment` 默认设为 true (`parser.set_defaults(asap_augment=True)`)。

22. **Manifest 何时更新，如何保证读者不会看到半成品？** 正常提交顺序是：row 被接受时 `OnlineFrameArchive.stream_frame` 就按帧写出 RGB-D 文件，`OnlineFrameArchive.archive_chunk` 在 `materialize` 时校验本 chunk 的帧文件齐全（不重写）；`ChunkDataWriter.commit_chunk_data` 再原子写 chunk pickle，原子更新聚合 `final_data/metadata`，最后原子更新 `online_data/manifest.json`；`commit` 返回后才调用 `OnlineFrameArchive.publish_metadata` 推进 archive 的 `metadata.json` 与 `enhance_metadata.json`。所以读者一旦从 manifest 看到新 committed chunk，对应 chunk 与帧文件已经存在；archive `frame_num` 只推进到已 commit 前缀。

`atomic_pickle_dump/atomic_json_dump` 都采用临时文件、flush、fsync 和 `os.replace`。RGB PNG 与 depth NPY 也都经 `atomic_open` 的 `temp+flush+fsync+os.replace` 写出，所以读者不会看到半写文件；未随任何 chunk 提交的 streamed 尾帧由 `discard_streamed_tail` 在收尾时删除。正常结束写 `finished`；`materialize/commit` try 块内的失败写 `failed`。更早的 `prepared-frame` 加载失败不在该 try 块内，因此不能声称任何异常都必然把 manifest 从 `recording` 改成 `failed`。

源码证据：

- `_materialize_and_commit_window` 明确执行 `archive_chunk` → `commit_chunk_data` → `publish_metadata`。
- `ChunkDataWriter.commit_chunk_data` 的顺序是 `atomic chunk` → `aggregate` → `counters` → `_write_manifest`。
- `OnlineFrameArchive.archive_chunk`、`stream_frame` 与 `publish_metadata` 说明 frame files 与 committed metadata 的关系。
- `atomic_pickle_dump/atomic_json_dump` 和 `atomic_open` 给出原子写细节；`ChunkStreamSession.run` 给出 `finished/failed` 的实际覆盖边界。

23. **训练侧从什么时候开始读取？** supervisor 可以在第一个 chunk 之前启动，但 Stage 1/train 不会对空数据创建 simulator/trainer。每个 stage 构造自己的 `OnlineChunkReader`，先等待 `online_data/manifest.json`，再通过 `wait_for_initial_frames` 循环读取，直到 `OnlineFrameBuffer.frame_len >= segment_len`。当前 chunk 是 5 帧，Stage 1 的 `segment_len=10`，所以至少等 2 chunks；train 的 `segment_len=30`，所以至少等 6 chunks。wrapper 按 Stage 1 → 可选 Stage 2 → train 顺序执行，因此 train 还必须等前面的启用阶段返回。

开始运行后，`OnlineChunkReader.load_new_chunks` 每次从 `last_loaded_chunk + 1` 顺序读到 manifest 的 `latest_committed_chunk`，再追加到 `session-lived OnlineFrameBuffer`；它不跳 chunk，也不把“最新一个”当作完整历史。`train.stop_when_finished: true` 时，trainer 以 manifest 的 `finished` 为停止条件，完成并保存观察到 `finished` 的 terminal iteration；设为 `false` 时严格跑 `iterations`。

源码证据：

- 外部 checkout 的 `optimize_online_cma.py::wait_for_initial_frames` 和 `train_online_warp.py::wait_for_initial_frames` 给出初始帧数 `gate`。
- 外部 `qgtd/data/online_stream.py::OnlineChunkReader.load_new_chunks` 与 `OnlineFrameBuffer.append_chunks` 给出连续游标和 growing buffer；
`qgtd/engine/trainer_warp.py::InvPhyTrainerWarp.train_online_batched` 给出持续刷新与 terminal save 行为。

Phystwin_shen 启动与数据交接 (Q24)

24. Phystwin_shen 如何启动，我们如何把数据传给它？ 启动条件。 只有

`downstream.mode=phystwin_shen` 才进入这条路径。

`OrchestratorRunConfig.from_args` (`orchestration/run_config.py`) 在 camera 启动前校验外部 checkout、pipeline config、conda env、stage window 参数和 viewer endpoint。运行中，`main.main::_ensure_phystwin_shen_running` 在每次 stream poll 前以及 chunk commit callback 中被调用：shape-prior warm-up 开启时，它等待 `<base_path>/shape_prior/points.npz`；该文件同时表示 prior 已完成且 SAM3D stage 子进程已释放 GPU。warm-up 关闭时则使用 `warmup_disabled_immediate` trigger。函数用 `phystwin_launch` is not None 保证整场只启动一次，后续调用只检查 supervisor 与 stdout relay 是否健康。

`launch_phystwin_shen` 在外部 repo 目录中用一个 Popen 启动 `scripts/run_online_full_pipeline.py`，设置 `CUDA_VISIBLE_DEVICES`、`PYTHONUNBUFFERED=1` 和 `start_new_session=True`。Demo 显式传入 `--online_dir <base_path>/online_data`，以及本地 `config/default.yaml::phystwin_shen` 的全部 runtime 叶子；外部 pipeline YAML 不再是这些 override 的维护源。当前 Stage 1 是 `batch/segment/stride=2/10/10`、`max_iter=2`、`cma_popsiz=4`、`boba/gather`；train 是 `5/30/30`、100 iterations。wrapper 先启动 `cma_viewer.source=all` 的 combined HTML viewer，再顺序启动 Stage 1、可选 Stage 2 和 train。supervisor 及 children 继承同一进程组；Demo 保存 PGID，用于异常或取消时整体停止。

数据交接。 这里没有 socket、HTTP upload、RPC、stdin pickle 或跨进程 Python queue。两个仓库通过同一台机器上的共享目录协议交接：

```

camera process
-> capture/frames.jsonl + capture/prepared_phystwin/*.npz
-> Demo chunk bridge (跟踪、ASAP、5 帧 window)
-> outputs/online_data/
    chunks/chunk_000000.pkl, chunk_000001.pkl, ...
    manifest.json
    color/0/*.png + depth/0/*.npz
    calibrate.pkl + metadata.json + enhance_metadata.json
-> Phystwin_shen OnlineChunkReader
-> OnlineFrameBuffer -> Stage 1 / Stage 2 / train GPU tensors

```

每个 window 完成后，Demo 先归档 RGB-D，再原子写 `chunks/chunk_{id:06d}.pkl` 和聚合 `data/final_data.pkl`，最后原子推进

`online_data/manifest.json::latest_committed_chunk`。因此 manifest 是 commit point: Phystwin 只读取它已经公布的连续 chunk，不会看到半写 pickle；若 manifest 声称某个 chunk 已提交但文件缺失，reader 立即抛错，而不是跳过。producer 不等待 consumer ACK；下游变慢时，chunk 保留在磁盘，reader 之后按游标补读。

chunk 中传递给 OnlineFrameBuffer 的时间序列字段是 `object_points`、`object_colors`、`object_visibilities`、`object_motions_valid`、`controller_points`、`asap_surface_points`、`asap_interior_points` 和 `source_frame_indices`。buffer 校验 frame range 连续、每帧 shape 稳定，再 concatenate 并转成目标 GPU tensor；`structure_points` 由首帧的 object、ASAP surface 和 ASAP interior 拼接。相机内外参与 `calibrate.pkl` 和 `metadata.json` 独立读取。触发启动的 `shape_prior/points.npz` **不是**主传输通道；正式 shape-prior trajectory 已经以每帧 ASAP 字段进入 chunk。Demo 不向这些相机 metadata 注入版本名、case 名、depth-encoding 描述或其他无 reader 的 provenance 字段；这避免把日志/诊断信息误当成跨仓库数据合同。

supervisor、Stage 1/2 和 train 的 stdout/stderr 由 Demo relay 同时写到启动终端（每行前缀 `[phystwin_shen]`）和

`<base_path>/phystwin_shen/online_full_pipeline.log`。producer 结束后 Demo 还会等待 supervisor；只有其 return code 0 才算完整 pipeline 成功，非零退出、relay failure 或上游异常都会终止整个保存的进程组。

源码证据：

- `main.main::_ensure_phystwin_shen_running` 给出唯一启动 gate、health check 与 points-ready trigger。
- `build_full_pipeline_command` 构造显式 `--online_dir/runtime CLI`；`launch_phystwin_shen` 给出 cwd、env、process group 和 output relay。

- `_materialize_and_commit_window` 组装正式 window;
`ChunkDataWriter.commit_chunk_data` 和 `OnlineFrameArchive.archive_chunk` 给出 chunk/archive/manifest 的生产端提交协议。
- 外部 checkout 的
`qqtt/data/online_stream.py::OnlineChunkReader/OnlineFrameBuffer` 给出消费端 poll、连续性/shape 校验、补读与 CPU→GPU 转换;
`optimize_online_cma.py::load_camera_metadata` 和
`train_online_warp.py::load_camera_metadata` 读取相机 metadata。

在线流水线状态可视化 (Q25)

25. 如何看到流水线当前在做什么，以及 warm-up 是否失败？ `PipelineStatusWriter.emit` 把 `t/source/stage/detail/ok` 追加到 `<base_path>/pipeline_status.jsonl`，并吞掉所有写入异常，所以 telemetry 失败不会改变正式产品。当前实际 writer 只有两类：orchestrator 写 run start、chunk committed、Phystwin downstream start 和正常控制流末尾的 finished/fatal；camera 写 capture start、shape-prior submitted、warm-up ready，以及 startup/worker fatal。shape-prior 的 upscale/generate/align/ sample 子进程没有各自创建 writer，也没有逐阶段状态事件。

`viz_playback.run_side_by_side` 会读取最近 200 条事件，并调用 `viz_panels.draw_pipeline_status` 绘制状态条；任一 `ok=False` 或 `STAGE_FATAL` 事件会令状态条变红并显示错误。但当前默认配置是 side-by-side + sam3d-final-data，`visualization.visualize_track.run` 会分派到 `run_interactive_side_by_side`；该函数目前没有读取状态日志，也没有绘制状态条。因此当前可核验的查看方式是：直接查看 `pipeline_status.jsonl`，或使用会进入 OpenCV `run_side_by_side` 的 `rgb-overlay` 路径；不能声称默认正式 viewer 已经显示该状态条。

`Phystwin_shen` 的 `http://127.0.0.1:8765/` 是另一套 combined HTML viewer。它展示 origin、Stage 1 和 train 的 realtime 结果，但不读取 `pipeline_status.jsonl`，也不替代上述 warm-up/fatal 状态条。它只在 points-ready 后由 supervisor 启动；Chrome 打开 URL 只是连接现有 server，不会反向启动 Phystwin pipeline。2026-07-12 正式 fake-live 启动中，该 endpoint 已实测返回 HTTP 200 并成功在现有 Chrome 会话中打开。

`main.main` 的外层 try/except/finally 覆盖 camera 启动、stream、supervisor launch 和最终 wait。shape-prior/materialization、camera、supervisor 非零退出或 Ctrl+C 任一失败都会写 terminal `STAGE_FATAL`，并以保存的 PGID 对整个 `Phystwin_shen` 进程组执行 SIGTERM，超时后 SIGKILL。

显式 `--max-chunks N` 达标时，stream 已经提交完第 N 个 chunk 并持久化 terminal `manifest.status=finished`。编排器会在等待 `Phystwin_shen` 继续训练之前立即、只打印一次：


```
#####  
collect finish  
#####
```

这个 banner 只表示采集完成，不表示整个 Demo 进程已经退出；Phystwin_shen 尚未训练完时，主进程会继续等待它。

Warm-up 实时 RGB 输入预览 (mdp.warmup_preview.WarmupRgbPreview, 不是 tracking-chunk viewer)：无论 downstream.mode 选什么，camera 进程在 capture 启动时打开一个实时 RGB 输入窗口（直接读内存里的 input_preview_slot，零磁盘 IO），供操作员在 hold-still 期间确认取景/双手可见。生命周期：warm-up 正常结束（WARMUP_FINISHED banner 处；若 shape-prior warm-up 关闭则在 frame-0 seed 完成处）自动关闭；warm-up 失败/取消/提前退出经 stop_event + stop() **立即**关闭。GUI 失败（无显示环境）只打一行日志并禁用，绝不影响采集。开关：--warmup-rgb-preview / --no-warmup-rgb-preview（默认开，编排器透传给 camera 子进程）。即使 supervisor leader 已退出，遗留的 train/viewer child 仍按保存 PGID 清理。reader 仍只把 manifest.status=finished 当完成；若 producer 写 failed，Demo 不等待 reader 自己理解 failed，而是由上述异常路径终止整个 downstream group。

源码证据：

- PipelineStatusWriter.emit 是 best-effort append；read_status_events 容忍缺失文件和 torn last line。
- main.main、MainDataProcessingDemo.run、FatalErrorLatch.record、main_data_processing.py 的 startup fatal handler，以及 ShapePriorPublisher 的 submitted/warm-up-ready 方法 是全仓实际 emit call sites；shape-prior stage 文件没有 writer call site。
- viz_playback.run_side_by_side 调用 draw_pipeline_status；use_interactive_side_by_side 与 run_interactive_side_by_side 证明默认 SAM3D viewer 绕过该绘制逻辑。
- config/default.yaml 定义默认 side-by-side + sam3d-final-data；ShapePriorWarmupManager._run 与 main.main 给出 terminal fatal 与 downstream PGID 清理边界。